

SmartPark AI

TECHNICAL PROJECT SUMMARY & VERIFICATION REPORT

1. Project Overview

SmartPark AI is an advanced, AI-powered smart parking management system architected to solve modern urban parking inefficiencies. By integrating cutting-edge computer vision, real-time IoT communication protocols, and modern web technologies, the platform delivers automated multi-occupancy tracking, instantaneous slot booking processing, and predictive route metrics for seamless parking space optimization.

2. Tech Stack

Frontend	React 18, Vite, Tailwind CSS, React Router, Axios, Leaflet (Interactive Maps), Framer Motion (Animations)
Backend	Node.js, Express, MongoDB, MQTT Broker, Socket.IO (Real-time events), Swagger (API Documentation)
AI Service	Python, FastAPI, OpenCV (Computer Vision), Pydantic (Data validation)
Database	MongoDB (NoSQL Document Store)
Deployment	Windows-compatible environment configuration, structured for seamless local service setup

3. Key Features

- **User Authentication & Dashboard:** Secure login infrastructure rendering personalized metrics, reservation history, and fast-track actions.
- **Live Geolocation Parking Map:** Interactive mapping system displaying instant real-time vehicle occupancy and open slot counts via visual status updates.
- **Booking & Slot Management:** End-to-end transactional pipeline allowing users to browse, hold, and manage active parking reservations seamlessly.
- **Administrative Controls:** Complete command center giving site managers full visibility over slot allocations, system telemetry, and user logs.
- **AI Analytics Infrastructure:** Core automated microservices handling image-based parking slot detection and intelligent route destination forecasting.

4. Running the App Locally

Follow this exact execution order to instantiate the local development environment:

Step 1: Launch local `MongoDB` instance and initialize the `MQTT` broker.

Step 2: Run the primary backend application server at: `http://127.0.0.1:5000`

Step 3: Run the compiled frontend development server at: `http://127.0.0.1:5174`

Step 4: Launch the Python machine learning service at: `http://127.0.0.1:8000`

5. Project Status & Integration Verification

- ✓ **Frontend Architecture Verified:** UI components, responsive layout matrices, and routing workflows have passed cross-browser consistency validation.
- ✓ **Backend Engine Verified:** Core API controllers, data persistence schemes, and real-time socket connections are operating within standard performance benchmarks.
- ✓ **AI Computer Vision Service Verified:** FastAPI endpoints reliably interpret model objects, return image matrices, and serve operational statistics.
- ✓ **End-to-End Authentication Verified:** The credential processing pipeline handles user session tokens and routes successfully to the primary `/app` dashboard layout.

6. Critical System Notes

- **Dashboard Routing:** Upon successful user authentication, session middleware routes automatically to the core dashboard view at `/app`.
- **API Orchestration Base URL:** All client queries communicate directly with the backend gateway routed through `http://localhost:5000/api`.
- **AI Diagnostics:** The automated machine learning microservice exposes an instantaneous health and uptime checkpoint at the root endpoint: `/`.

Conclusion: SmartPark AI successfully unifies high-performance computer vision pipelines with low-latency communication middleware to provide a scalable framework for smart city parking management. All distributed services are validated, stable, and ready for deployment.